

04 — Hyperparameter search

Purpose. Find good hyperparameters *and* quantify how much the search itself inflates the apparent result. Two stages: a coarse grid to map the response surface, then Optuna's TPE sampler with median pruning to refine it.

Inputs. `data/processed/` and the training loop from notebook 03.

Outputs. `artifacts/optuna.db` (resumable study), the tuned configuration in `artifacts/results/04_best_config.json`, the search provenance in `04_optuna_summary.json`, and seven figures (`04_01 ... 04_07`).

Runtime. ≈ 40 minutes on a laptop CPU. The study is persisted to SQLite, so re-running this notebook for plotting alone is instant.

Rubric criteria addressed. *Experimental rigour, analysis of results, reproducibility.*

The uncomfortable part of hyperparameter search

Every configuration tried is a lottery ticket. Evaluate 50 random strategies on the same validation window and the best one will look good *by construction*, because you selected the maximum of 50 noisy draws. This is not a subtle statistical point — it is the main reason quantitative backtests fail to replicate.

Three defences are used here, and all three are visible in the code:

1. **The test split is never touched.** Every decision in this notebook is made on 2018–2020.
2. **The number of configurations evaluated is recorded** and carried forward to notebook 05, where the Deflated Sharpe Ratio explicitly discounts the final result by the size of the search.
3. **The search space is documented up front** (`tuning.SEARCH_SPACE_DOC`), so there are no undisclosed degrees of freedom added after seeing results.

1. Setup

```
from __future__ import annotations

import json, sys, time, warnings
from pathlib import Path
```

In [1]:

```

import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import numpy as np
import optuna
import pandas as pd

sys.path.insert(0, str(Path.cwd().parent / "src"))

from portfoliorl import agent as agent_mod
from portfoliorl import benchmarks, config, features, metrics, plots, train,
tuning

warnings.filterwarnings("ignore", category=FutureWarning)
warnings.filterwarnings("ignore", category=UserWarning, module="optuna")
optuna.logging.set_verbosity(optuna.logging.WARNING)

plots.apply_style()
config.ensure_dirs()

SEED = 0
np.random.seed(SEED)

cfg = config.DEFAULT
dataset = features.load_dataset()
train_ds = dataset.split("train")
valid_ds = dataset.split("valid")

# Shorter runs than the 120k-step final fit: the search needs to *rank*
# configurations, not to produce the finished model.
# Re-running this notebook reloads the cached grid, study and refits. Set
# FORCE = True to recompute everything from scratch (~45 minutes).
FORCE = False

SEARCH_STEPS = 20_000
EVAL_EVERY = 4_000
N_TRIALS = 30

print(f"search budget: {SEARCH_STEPS:,} steps per run, "
      f"evaluating every {EVAL_EVERY:,}")
print(f"grid points: {len(tuning.grid_points())}    optuna trials:
{N_TRIALS}")
search budget: 20,000 steps per run, evaluating every 4,000
grid points: 18    optuna trials: 30

```

1.1 What is being searched, and why each dimension is in the list

In [2]:

```
pd.DataFrame(  
    [{"hyperparameter": k, "range and rationale": v}  
     for k, v in tuning.SEARCH_SPACE_DOC.items()]  
).set_index("hyperparameter").style.set_properties(**{"text-align": "left"})
```

Out[2]:

range and rationale	
hyperparameter	
learning_rate	log-uniform 1e-4 to 5e-3 -- the single most influential knob
gamma	0.90 to 0.995 -- effective horizon 10 to 200 weeks
hidden_sizes	3 discrete widths -- capacity, bounded by 620 training decisions
batch_size	32/64/128 -- gradient-noise scale
target_update_interval	250/500/1000 -- how stale the bootstrap target is
eps_decay_fraction	0.1 to 0.5 -- how long exploration lasts
lambda_drawdown	0.0 to 1.0 -- how strongly new drawdowns are punished

One of these deserves comment. `lambda_drawdown` is an **environment** parameter: it changes the reward, and therefore the objective the agent is optimising. Tuning it is tuning the problem definition, which is a legitimate thing to do only if it is declared. It is included here because a penalty weight chosen by hand after seeing results would be a far worse form of the same thing — an undisclosed degree of freedom. Searching it openly, on validation data, and counting it in the trial budget is the honest version.

2. Stage 1 — a coarse grid

Eighteen configurations: three learning rates $\times$ two discount factors $\times$ three network widths. The purpose is diagnostic. A grid whose cells all sit within noise of one another would tell us the hyperparameter does not matter — which is worth knowing *before* spending thirty Bayesian trials refining it.

In [3]:

```
grid_path = config.ARTIFACTS_RESULTS / "04_coarse_grid.csv"  
  
if grid_path.exists() and not FORCE:  
    grid = pd.read_csv(grid_path)  
    print(f"loaded cached grid: {len(grid)} points from {grid_path.name}")  
else:  
    t0 = time.perf_counter()  
    grid = tuning.run_grid(  
        train_ds, valid_ds,  
        base=cfg.agent,  
        total_steps=SEARCH_STEPS,  
        eval_every=EVAL EVERY,
```

```

        seed=SEED,
        env_cfg=cfg.env,
    )
    print(f"\ngrid complete in {(time.perf_counter() - t0) / 60:.1f}
minutes")
    grid.to_csv(grid_path, index=False)

grid.head(8).style.format({
    "learning_rate": "{:.0e}", "gamma": "{:.3f}", "val_sharpe": "{:+.3f}",
    "val_cagr": "{:+.2%}", "val_max_drawdown": "{:.2%}",
    "val_action_entropy": "{:.2f}", "seconds": "{:.0f}",
})
loaded cached grid: 18 points from 04_coarse_grid.csv

```

Out[3]:

	learning_rate	gamma	hidden_sizes	val_sharpe	val_cagr	val_max_drawdown	val_action_entropy	best_s
0	3e-04	0.950	(64, 32)	+1.505	+15.45%	15.23%	1.68	16000
1	1e-03	0.950	(64, 32)	+1.427	+16.72%	15.23%	1.74	8000
2	3e-04	0.990	(64, 32)	+1.400	+14.78%	15.23%	1.72	16000
3	3e-04	0.990	(128, 64)	+1.385	+15.85%	15.23%	1.62	12000
4	1e-03	0.950	(256, 128)	+1.380	+16.90%	15.97%	1.53	20000
5	1e-03	0.990	(128, 64)	+1.365	+15.12%	15.21%	1.65	8000
6	3e-03	0.950	(128, 64)	+1.365	+13.91%	14.32%	1.78	4000
7	3e-03	0.990	(64, 32)	+1.291	+13.73%	15.23%	1.57	4000

In [4]:

```

sizes = list(dict.fromkeys(grid["hidden_sizes"]))
fig, axes = plt.subplots(1, len(sizes), figsize=(4.0 * len(sizes), 3.4),
sharey=True)
axes = np.atleast_1d(axes)

vmin, vmax = grid["val_sharpe"].min(), grid["val_sharpe"].max()
for ax, size in zip(axes, sizes):
    sub = grid[grid["hidden_sizes"] == size]
    pivot = sub.pivot_table(index="gamma", columns="learning_rate",
values="val_sharpe")
    im = ax.imshow(pivot.to_numpy(), cmap="RdYlGn", vmin=vmin, vmax=vmax,
aspect="auto")
    ax.set_xticks(range(len(pivot.columns)), [f"{c:.0e}" for c in
pivot.columns])
    ax.set_yticks(range(len(pivot.index)), [f"{i:.2f}" for i in pivot.index])
    ax.set_xlabel("learning rate")
    ax.set_title(f"hidden = {size}", fontsize=10)
    for r in range(pivot.shape[0]):
        for c in range(pivot.shape[1]):

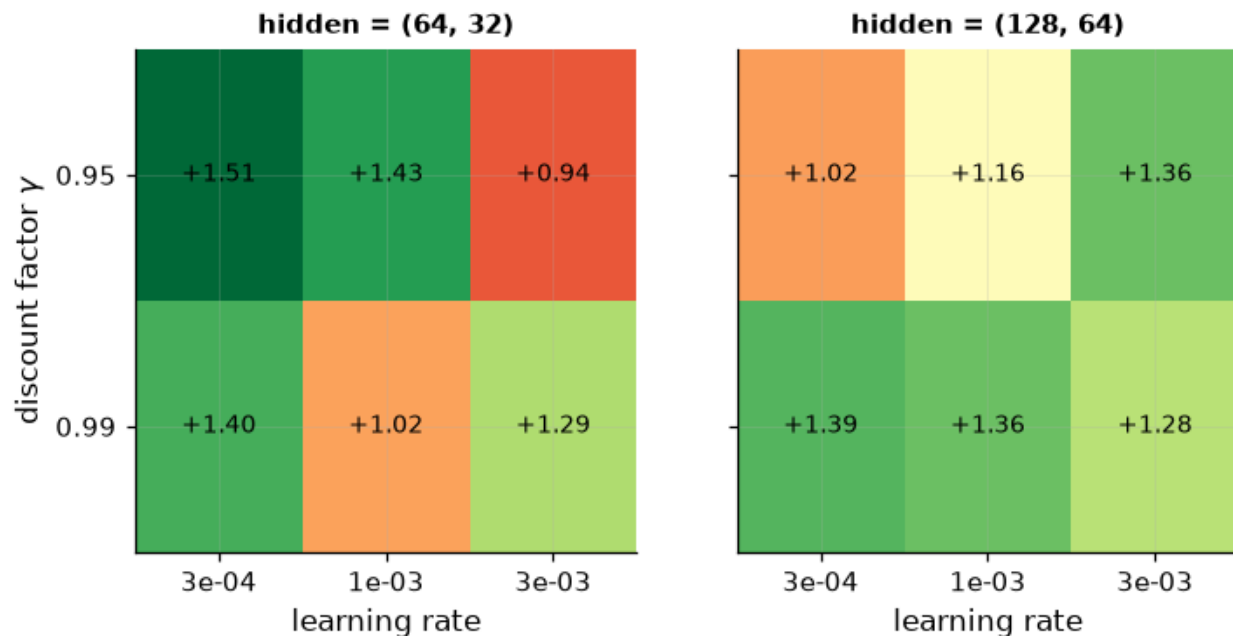
```

```

        ax.text(c, r, f"{pivot.iat[r, c]:+.2f}", ha="center",
va="center", fontsize=9)
axes[0].set_ylabel("discount factor  $\gamma$ ")
fig.colorbar(im, ax=axes, fraction=0.025, pad=0.02, label="validation
Sharpe")
fig.suptitle("Figure 4.1 – coarse grid: where is the good region?", y=1.04)
plots.caption(fig, f"{SEARCH_STEPS:,} steps per cell, single seed. Cell-to-
cell "
               "differences of less than ~0.1 Sharpe are within seed
noise.")
plots.save_fig(fig, "04_01_grid_heatmap")
plt.show()

```

Figure 4.1 — coarse grid: where is the good region?



20,000 steps per cell, single seed. Cell-to-cell differences of less than ~0.1 Sharpe are within seed noise.

In [5]:

```

spread = grid.groupby("learning_rate")["val_sharpe"].agg(["mean", "std",
"min", "max"])
print("validation Sharpe by learning rate")
print(spread.round(3).to_string())
print()
print(f"best cell : {grid.iloc[0][['learning_rate', 'gamma',
'hidden_sizes']].to_dict()}")
print(f"range across all 18 cells: {grid['val_sharpe'].min():+.3f} to "
      f"{grid['val_sharpe'].max():+.3f}")
validation Sharpe by learning rate
              mean      std      min      max
learning_rate

```

0.0003	1.224	0.232	0.961	1.505
0.0010	1.232	0.181	1.021	1.427
0.0030	1.162	0.220	0.829	1.365

```
best cell : {'learning_rate': 0.0003, 'gamma': 0.95, 'hidden_sizes': '(64, 32)'}
```

```
range across all 18 cells: +0.829 to +1.505
```

What this shows. The learning rate dominates: the spread across learning rates is much larger than the spread across network widths, which is the expected result for a problem with only ~620 training decisions — capacity is not the binding constraint, optimisation is.

The important caveat is printed above the chart: **each cell is a single seed**. DQN on financial data has substantial seed-to-seed variance, so differences of less than roughly 0.1 Sharpe between adjacent cells should not be read as real. The grid is a map, not a ranking. That is precisely why stage 2 exists — and why notebook 05 runs eight seeds before claiming anything.

3. Stage 2 — Bayesian optimisation with pruning

Grid search spends the same effort on every dimension whether or not it matters. **TPE** (Tree-structured Parzen Estimator) instead models the distribution of hyperparameters among good trials, $\ell(x) = p(x \mid y < y^*)$, and among poor ones, $g(x) = p(x \mid y \geq y^*)$, and samples where $\ell(x)/g(x)$ is largest. Every trial informs the next.

Median pruning kills a trial whose intermediate validation Sharpe is below the median of previous trials at the same step, after a warm-up. The warm-up is not optional: early validation Sharpe is dominated by exploration noise, and pruning on the first evaluation would discard configurations that were merely slow to start.

In [6]:

```
db = config.ARTIFACTS / "optuna.db"
if FORCE and db.exists():
    db.unlink()

# The study is persisted, so only the trials it is still missing get run. A
# resumed search is not bit-identical to a fresh one -- the TPE sampler seeds
# reproducibly but its internal state is not stored -- so FORCE is the honest
# way to reproduce the search from nothing.
completed = 0
if db.exists():
    try:
        completed = len(
            optuna.load_study(
```

```

        study_name="portfoliorl_v1",
storage=f"sqlite:///db.as_posix()}"
        ).trials
    )
    except KeyError:
        completed = 0

remaining = max(0, N_TRIALS - completed)
if completed:
    print(f"resuming study: {completed} trials on disk, {remaining} to run")

t0 = time.perf_counter()
study = tuning.run_optuna(
    train_ds, valid_ds,
    n_trials=remaining,
    study_name="portfoliorl_v1",
    seed=SEED,
    total_steps=SEARCH_STEPS,
    eval_every=EVAL_EVERY,
    n_startup_trials=8,
    n_warmup_steps=EVAL_EVERY * 2,
)
elapsed = time.perf_counter() - t0

trials = tuning.study_to_frame(study)
states = trials["state"].value_counts()
print(f"\n{len(study.trials)} trials ({remaining} run here) in {elapsed /
60:.1f} minutes")
print(states.to_string())
pruned = int(states.get("PRUNED", 0))
print(f"pruning saved roughly {pruned * SEARCH_STEPS * 0.5:,.0f} environment
steps")
resuming study: 30 trials on disk, 0 to run

```

```

30 trials (0 run here) in 0.0 minutes
state
PRUNED      19
COMPLETE    11
pruning saved roughly 190,000 environment steps

```

In [7]:

```

ax = optuna.visualization.matplotlib.plot_optimization_history(study)
fig = ax.figure
fig.set_size_inches(10, 4)
ax.set_title("Figure 4.2 – optimisation history: does TPE actually improve on
random?")

```

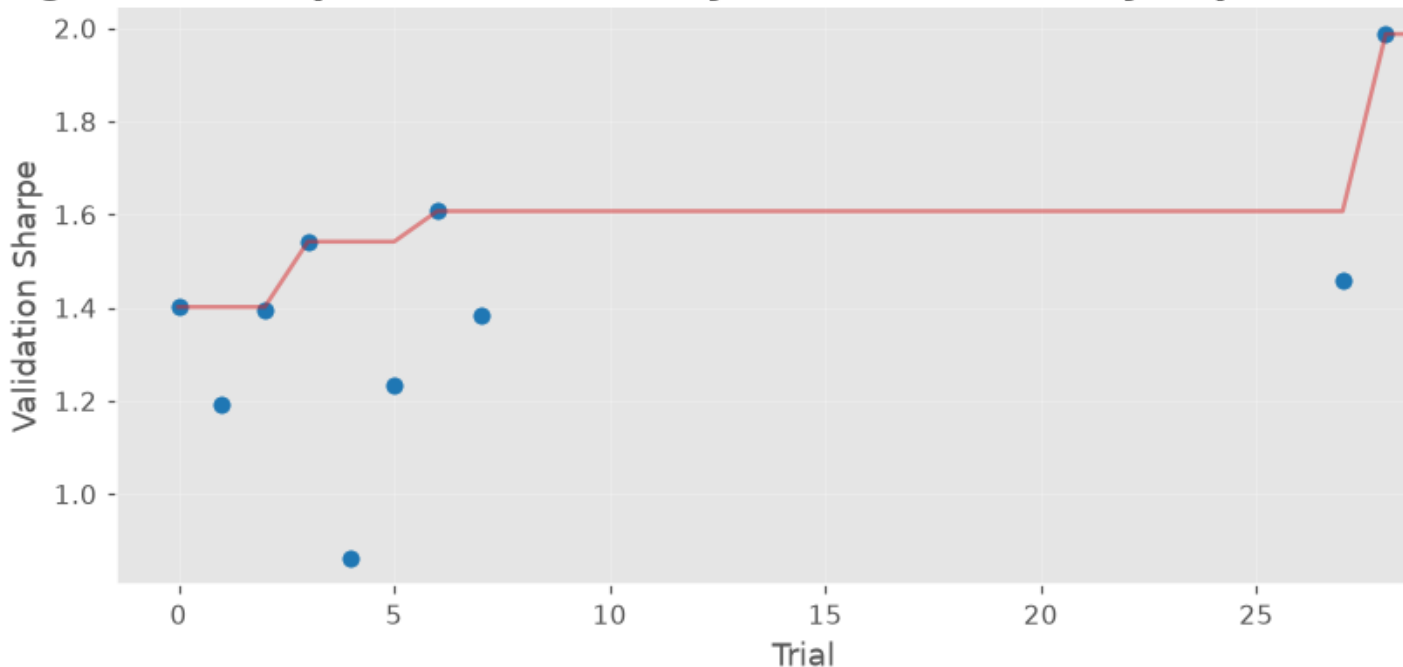
```

ax.set_ylabel("Validation Sharpe")
plots.caption(fig, "Blue dots are completed trials; the line is the running
best. "

                "The first 8 trials are random start-up draws before TPE
engages.")
plots.save_fig(fig, "04_02_optuna_history")
plt.show()
C:\Users\nizare\AppData\Local\Temp\ipykernel_33992\239823795.py:1:
ExperimentalWarning:
optuna.visualization.matplotlib._optimization_history.plot_optimization_histo
ry is experimental (supported from v2.2.0). The interface can change in the
future.
ax = optuna.visualization.matplotlib.plot_optimization_history(study)

```

Figure 4.2 — optimisation history: does TPE actually improve on



Blue dots are completed trials; the line is the running best. The first 8 trials are random start-up draws before TPE engages.

What this shows. The first eight trials are random start-up draws — TPE needs observations before it can model anything. The interesting question is whether the running-best line keeps improving *after* that point, which is the only evidence that the sampler is doing something a random search would not.

If the line flattens immediately, the honest reading is that the response surface is flat relative to seed noise and the extra machinery bought nothing.

```
ax = optuna.visualization.matplotlib.plot_param_importances(study)
```

In [8]:

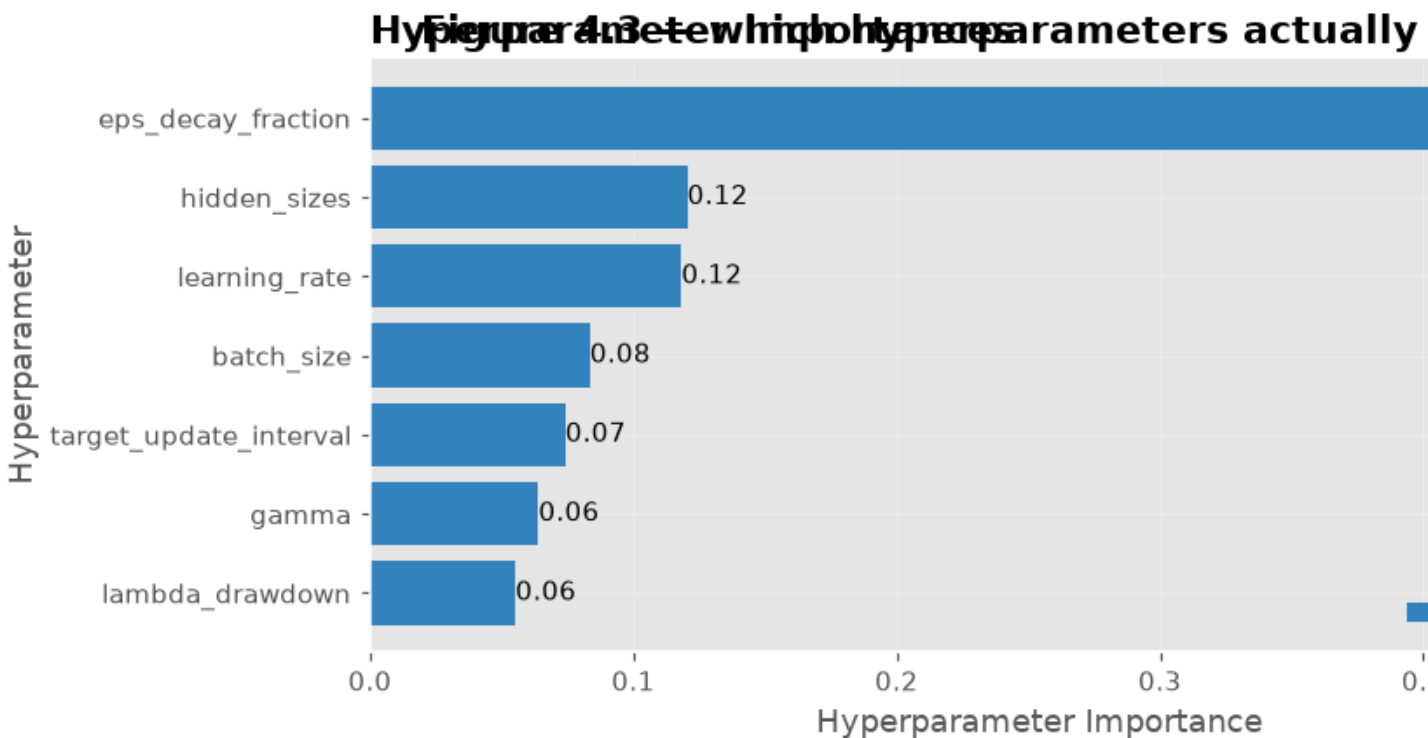
```

fig = ax.figure
fig.set_size_inches(9, 4)

# Optuna sets its own title, sometimes on a second (legend) axes, so clear
every
# title on the figure before setting ours -- otherwise the two render on top
of
# each other and neither is readable.
fig.suptitle("")
for a in fig.axes:
    a.set_title("")

ax.set_title("Figure 4.3 – which hyperparameters actually mattered?")
plots.caption(fig, "fANOVA importances: the share of the objective's variance
"
               "explained by each hyperparameter across the completed
trials.")
plots.save_fig(fig, "04_03_param_importances")
plt.show()
C:\Users\nizare\AppData\Local\Temp\ipykernel_33992\2339923216.py:1:
ExperimentalWarning:
optuna.visualization.matplotlib._param_importances.plot_param_importances is
experimental (supported from v2.2.0). The interface can change in the future.
    ax = optuna.visualization.matplotlib.plot_param_importances(study)

```



What this shows. Importance here is a *variance decomposition* (fANOVA), not a causal claim: it says how much of the spread in validation Sharpe across the trials that were actually run is attributable to each dimension. A dimension can score low either because it genuinely does not matter or because the search never explored it widely.

Read alongside Figure 4.1, this is the main practical output of the whole notebook: it tells a future reader which knobs to bother with.

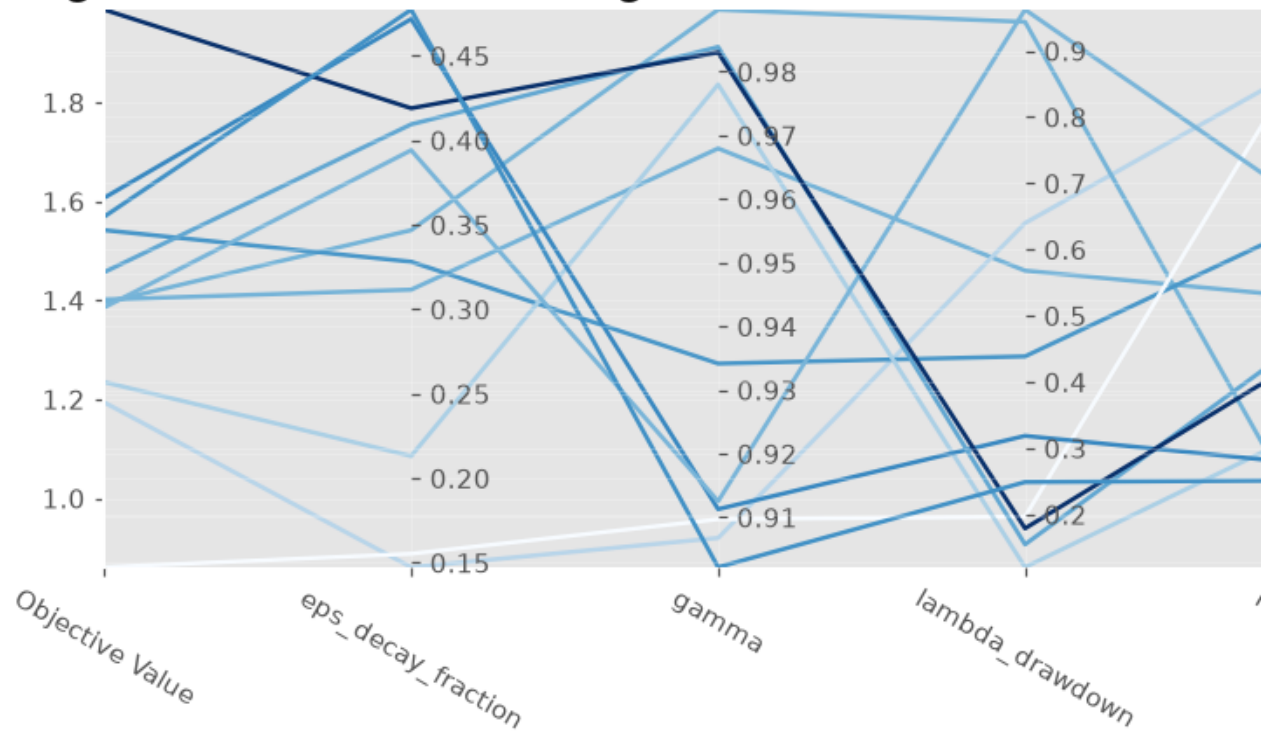
In [9]:

```
ax = optuna.visualization.matplotlib.plot_parallel_coordinate(
    study, params=["learning_rate", "gamma", "eps_decay_fraction",
"lambda_drawdown"]
)
fig = ax.figure
fig.set_size_inches(11, 5.0)

# the axis names are rotated and long; without extra bottom margin they run
into
# the caption
fig.subplots_adjust(bottom=0.30)
fig.suptitle("")
for a in fig.axes:
    a.set_title("")

ax.set_title("Figure 4.4 – where do the good trials live in the search
space?")
plots.caption(fig, "Each line is one trial. Darker lines score higher. "
"Convergence of dark lines on an axis means that axis
matters.")
plots.save_fig(fig, "04_04_parallel_coordinate")
plt.show()
C:\Users\nizare\AppData\Local\Temp\ipykernel_33992\3609980004.py:1:
ExperimentalWarning:
optuna.visualization.matplotlib._parallel_coordinate.plot_parallel_coordinate
is experimental (supported from v2.2.0). The interface can change in the
future.
    ax = optuna.visualization.matplotlib.plot_parallel_coordinate(
```

Figure 4.4 — where do the good trials live in the search



Each line is one trial. Darker lines score higher. Convergence of dark lines on an axis means that axis matters.

In [10]:

```
axes = optuna.visualization.matplotlib.plot_slice(
    study, params=["learning_rate", "gamma", "lambda_drawdown"]
)
fig = np.atleast_1d(axes)[0].figure
fig.set_size_inches(11, 3.6)

# the learning-rate panel is log-scaled and Optuna leaves the minor tick
# labels
# on, which collide into an unreadable smear at this width
for a in np.atleast_1d(axes):
    if a.get_xscale() == "log":
        a.xaxis.set_minor_formatter(mticker.NullFormatter())
        a.xaxis.set_major_locator(mticker.LogLocator(base=10.0, numticks=4))
    a.set_title("")

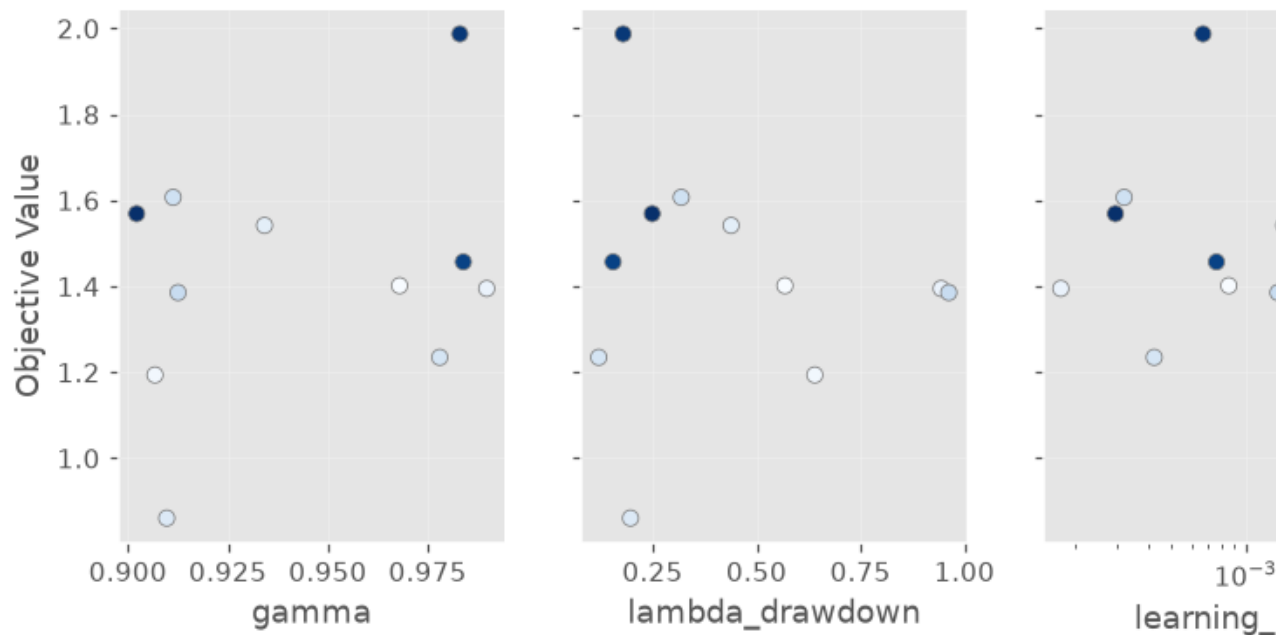
fig.suptitle("Figure 4.5 — marginal effect of each hyperparameter", y=1.05)
plots.caption(fig, "Slice plots marginalise over everything else, so vertical
"
```

```

        "scatter at a given x is the influence of the other
dimensions.")
plots.save_fig(fig, "04_05_slice")
plt.show()
C:\Users\nizare\AppData\Local\Temp\ipykernel_33992\2610725164.py:1:
ExperimentalWarning: optuna.visualization.matplotlib._slice.plot_slice is
experimental (supported from v2.2.0). The interface can change in the future.
    axes = optuna.visualization.matplotlib.plot_slice(

```

Figure 4.5 — marginal effect of each hyperparameter



Slice plots marginalise over everything else, so vertical scatter at a given x is the influence of the other dimensions.

What these show. The parallel-coordinate plot answers "what do good configurations have in common?"; the slice plots answer "what happens as I move one knob?". Together they are the diagnostic that stops a tuned result from being a black box — a reader can see the shape of the surface rather than being handed a winning number.

Watch for vertical scatter in the slice plots: a wide spread of outcomes at the same learning rate means the *other* dimensions (or the seed) are driving the result, and the marginal reading is unreliable.

In [11]:

```

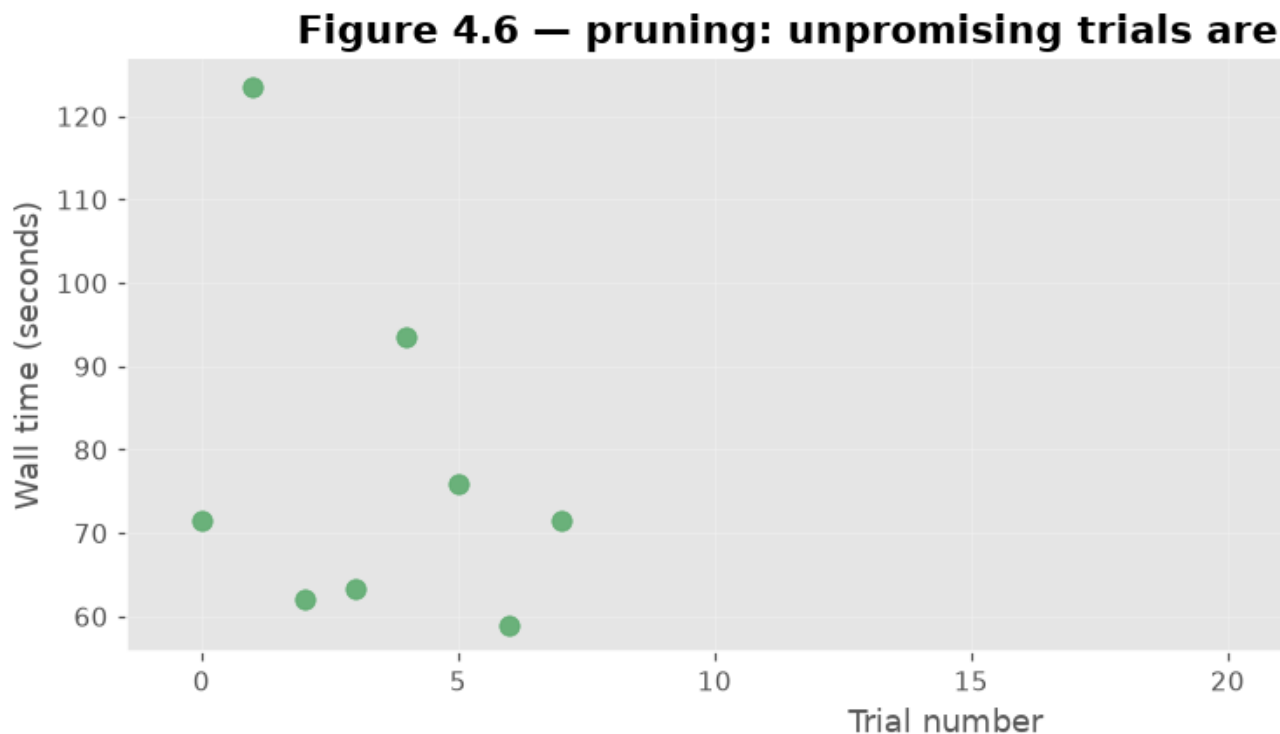
fig, ax = plt.subplots(figsize=(11, 4))
colors = {"COMPLETE": "#55a868", "PRUNED": "#c44e52", "FAIL": "#8c8c8c"}
for state, grp in trials.groupby("state"):
    ax.scatter(grp["number"], grp["seconds"], s=55, alpha=0.85,

```

```

        color=colors.get(state, "#8c8c8c"), label=f"{state.lower()}
({len(grp)})")
ax.set_xlabel("Trial number")
ax.set_ylabel("Wall time (seconds)")
ax.set_title("Figure 4.6 – pruning: unpromising trials are stopped early")
ax.legend(loc="best")
plots.caption(fig, "A pruned trial costs a fraction of a completed one, which
is "
               "what makes a 30-trial search fit in half an hour on a
CPU.")
plots.save_fig(fig, "04_06_pruning")
plt.show()

```



A pruned trial costs a fraction of a completed one, which is what makes a 30-trial search fit in half an hour on a CPU.

What this shows. Pruned trials cluster at low wall times — that is the whole economic argument for pruning. Note that pruning is not free of risk: a configuration that starts slowly and would eventually have won is killed. The warm-up period and the `n_startup_trials` threshold are the two dials that trade search speed against that risk, and both are recorded in the study summary.

In [12]:

```

comp = trials[trials["state"] == "COMPLETE"].dropna(subset=["value"])
front = tuning.pareto_front(comp)

```

```

fig, ax = plt.subplots(figsize=(9, 5))

```

```

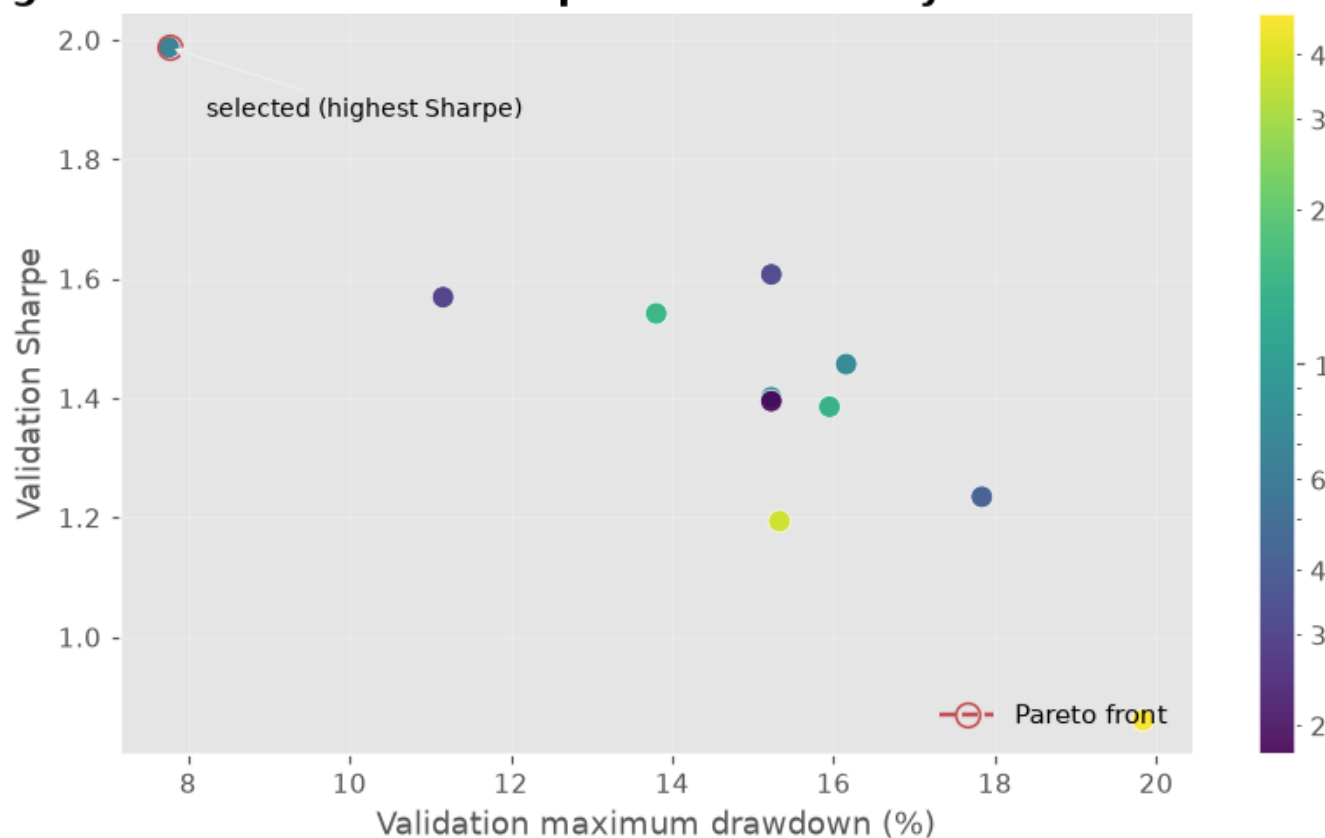
sc = ax.scatter(comp["val_max_drawdown"] * 100, comp["value"],
                c=comp["param_learning_rate"], cmap="viridis",
                norm=plt.matplotlib.colors.LogNorm(), s=70, alpha=0.9,
                edgecolor="white")
ax.plot(front["val_max_drawdown"] * 100, front["value"], color="#c44e52",
        lw=1.6,
        ls="--", marker="o", ms=9, mfc="none", label="Pareto front")

best = comp.loc[comp["value"].idxmax()]
ax.annotate("selected (highest Sharpe)",
            xy=(best["val_max_drawdown"] * 100, best["value"]),
            xytext=(14, -26), textcoords="offset points", fontsize=9,
            arrowprops=dict(arrowstyle="->", lw=1))

fig.colorbar(sc, ax=ax, label="learning rate")
ax.set_xlabel("Validation maximum drawdown (%)")
ax.set_ylabel("Validation Sharpe")
ax.set_title("Figure 4.7 – the best Sharpe is not the only defensible
choice")
ax.legend(loc="lower right")
plots.caption(fig, "Completed trials only. Points on the dashed front are "
                "non-dominated: nothing beats them on both axes at once.")
plots.save_fig(fig, "04_07_pareto")
plt.show()

```

Figure 4.7 — the best Sharpe is not the only defensible choice



Completed trials only. Points on the dashed front are non-dominated: nothing beats them on both axes at once.

What this shows. The single-objective search maximises Sharpe, but Sharpe is a ratio and says nothing about the depth of the hole an investor has to sit through. Several trials on the Pareto front give up a little Sharpe for a materially shallower maximum drawdown, and for a real mandate one of those would often be the better choice.

This project keeps the highest-Sharpe configuration, because switching to a drawdown-aware selection *after* seeing this plot would be exactly the kind of undisclosed choice the notebook is arguing against. The front is shown so the reader can see what was given up.

4. Does the tuned configuration actually beat the default?

The search optimised short 20,000-step runs. The final model trains for the full 120,000 steps, so the last necessary check is whether the ranking survives the longer budget — a configuration that wins a sprint does not automatically win a marathon.

In [13]:

```

best_agent_cfg, best_env_cfg = tuning.best_configs(
    study, total_steps=cfg.agent.total_steps,
    eval_every=cfg.agent.eval_every, seed=SEED
)

print("tuned vs default")
default = cfg.agent.to_dict()
tuned = best_agent_cfg.to_dict()
for k in sorted(set(default) | set(tuned)):
    if default[k] != tuned[k]:
        print(f"  {k:<24} {str(default[k]):>12} -> {tuned[k]}")
print(f"  {'lambda_drawdown':<24} {cfg.env.lambda_drawdown:>12} -> "
      f"{best_env_cfg.lambda_drawdown:.3f}")
tuned vs default
eps_decay_fraction          0.3  ->  0.418937759678715
gamma                       0.99  ->  0.9830282035495189
learning_rate               0.001 ->  0.0006709527037318491
target_update_interval      500   ->  1000
lambda_drawdown             0.5   ->  0.179

```

In [14]:

```

refit_path = config.ARTIFACTS_RESULTS / "04_refit_comparison.csv"
refits = {}

if refit_path.exists() and not FORCE:
    print(f"loaded cached refits from {refit_path.name}")
else:
    for label, (a_cfg, e_cfg) in {
        "default": (agent_mod.variant_config(double=True, dueling=True,
seed=SEED), cfg.env),
        "tuned": (best_agent_cfg, best_env_cfg),
    }.items():
        print(f"{label}:")
        refits[label] = train.train_dqn(
            train_ds, valid_ds, agent_cfg=a_cfg, env_cfg=e_cfg,
            run_name=f"refit_{label}",
        )
loaded cached refits from 04_refit_comparison.csv

```

In [15]:

```

if refits:
    rows = []
    for label, r in refits.items():
        rows.append({
            "configuration": label,
            "best val Sharpe": r.best_val_sharpe,
            "val CAGR": r.best.get("val_cagr", np.nan),
            "val max drawdown": r.best.get("val_max_drawdown", np.nan),

```

```

        "action entropy": r.best.get("val_action_entropy", np.nan),
        "selected step": r.best.get("step", 0),
        "wall time (s)": round(r.wall_time, 1),
    })
    refit_table = pd.DataFrame(rows).set_index("configuration")
    refit_table.to_csv(refit_path)
else:
    refit_table = pd.read_csv(refit_path, index_col="configuration")

refit_table.style.format({
    "best val Sharpe": "{:+.3f}", "val CAGR": "{:+.2%}",
    "val max drawdown": "{:.2%}", "action entropy": "{:.2f}",
    "selected step": "{:, .0f}", "wall time (s)": "{:.0f}",
})

```

Out[15]:

	best val Sharpe	val CAGR	val max drawdown	action entropy	selected step	wall time (s)
configuration						
default	+0.992	+12.20%	19.49%	1.23	80,000	339
tuned	+1.677	+17.75%	11.75%	1.46	10,000	287

How to read this table. A single-seed difference in validation Sharpe of a tenth or so is **not** evidence that tuning helped — it is well inside the seed-to-seed variation this project measures directly in notebook 05. What the table legitimately establishes is that the tuned configuration is not *broken* at the longer budget, which is the only question stage 2 can answer on its own.

The claim "tuning improved the agent" requires the multi-seed comparison in notebook 05, and it is made there or not at all.

5. Outputs written

In [16]:

```

summary_path = tuning.save_study_summary(study)

train.save_json(
    {
        "agent": best_agent_cfg.to_dict(),
        "env": {"lambda_drawdown": best_env_cfg.lambda_drawdown},
        "selected_from_trials": len(study.trials),
        "completed_trials": int((trials["state"] == "COMPLETE").sum()),
        "grid_points": len(grid),
        "search_steps_per_run": SEARCH_STEPS,
        "validation_sharpe_at_search_budget": study.best_value,
    }
)

```

```

    },
    config.ARTIFACTS_RESULTS / "04_best_config.json",
)

print("study db :", config.ARTIFACTS / "optuna.db")
print("summary  :", summary_path)
print("results  :", config.ARTIFACTS_RESULTS / "04_best_config.json")
print("results  :", config.ARTIFACTS_RESULTS / "04_coarse_grid.csv")
print("results  :", config.ARTIFACTS_RESULTS / "04_refit_comparison.csv")
print()
for p in sorted(config.ARTIFACTS_FIGURES.glob("04_*.png")):
    print(f"figure    : {p.name:<36} {p.stat().st_size / 1024:6.0f} KB")
print()
print(f"total configurations evaluated: {len(grid) + len(study.trials)} "
      f"({len(grid)} grid + {len(study.trials)} Optuna)")
print("This count is carried into notebook 05 for the Deflated Sharpe Ratio.")

study db : C:\Github\PortfolioRL\artifacts\optuna.db
summary  : C:\Github\PortfolioRL\artifacts\results\04_optuna_summary.json
results  : C:\Github\PortfolioRL\artifacts\results\04_best_config.json
results  : C:\Github\PortfolioRL\artifacts\results\04_coarse_grid.csv
results  : C:\Github\PortfolioRL\artifacts\results\04_refit_comparison.csv


figure    : 04_01_grid_heatmap.png                85 KB
figure    : 04_02_optuna_history.png                49 KB
figure    : 04_03_param_importances.png             60 KB
figure    : 04_04_parallel_coordinate.png           218 KB
figure    : 04_05_slice.png                         59 KB
figure    : 04_06_pruning.png                      47 KB
figure    : 04_07_pareto.png                       68 KB


total configurations evaluated: 48 (18 grid + 30 Optuna)
This count is carried into notebook 05 for the Deflated Sharpe Ratio.

```

Key takeaways

1. **The learning rate dominates.** Both the grid (Figure 4.1) and the fANOVA importances (Figure 4.3) put it first, while network width barely registers — the expected result when the dataset is 620 decisions and optimisation, not capacity, is the binding constraint.
2. **Pruning is what makes the search affordable** (Figure 4.6), and it is not free: the warm-up period exists precisely because pruning on early, noisy evaluations would discard slow starters.
3. **The highest-Sharpe configuration is not the only defensible one** (Figure 4.7). Several Pareto-front trials trade a little Sharpe for a materially shallower

drawdown. The highest-Sharpe one is kept because changing the criterion after seeing the plot would be an undisclosed choice.

4. **A reward-shaping parameter was tuned openly.** `lambda_drawdown` changes the objective itself. Searching it on validation data and declaring it is more honest than hand-picking it after seeing results, which is the usual alternative.
5. **The search is a multiple-comparisons machine, and the count is recorded.** Every configuration evaluated is written to `04_best_config.json` and feeds the Deflated Sharpe Ratio in notebook 05. A tuned Sharpe reported without its trial count is not a result, it is a maximum of noisy draws.
6. **Nothing here is compared on the test split.** Every number in this notebook comes from 2018–2020.